F Y E O

Security Code Review of Watch.fun

Chillchat

June 2026
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

Table of Contents

Executive Summary

Overview

Chillchat engaged FYEO Inc. to perform a Security Code Review of Watch.fun.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on May 05 - May 12, 2026, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-WAT-01 – Full operator signature bypass via unchecked Ed25519 headers
- FYEO-WAT-02 – Cross-type signature replay via missing domain separation
- FYEO-WAT-03 – special_limit bypassable
- FYEO-WAT-04 – Special package buyers never receive special_bonus_ticket_count
- FYEO-WAT-05 – Admin mid-sale mutations can rewrite live drop and package state
- FYEO-WAT-06 – admin_close_account enables signature replay and per user cap bypass
- FYEO-WAT-07 – admin_set_winners ignores pending_winners making randomness advisory
- FYEO-WAT-08 – buy_package_gift payment signature does not bind recipient
- FYEO-WAT-09 – claim_gift reads a live ticket_count instead of snapshot
- FYEO-WAT-10 – select winner randomness handling gaps

- FYEO-WAT-11 – DropAcc::create missing param validation for id and tickets_per_entry
- FYEO-WAT-12 – No on-chain rotation path for admin, operator
- FYEO-WAT-13 – Unbounded Vec pushes exceed max_len allocations
- FYEO-WAT-14 – sig_receipt could be griefed

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Security Code Review of Watch.fun. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a private repository at https://gitlab.com/chillchat/watch.fun.contract with the commit hash ff42fb0c3f69063375de941b6be2645a956ff715.

Remediations were submitted with the commit hash b8fde3f87a42906ba6b0e495c325e63c97936c17.

Files included in the code review

```
watch.fun.contract/
└── programs/
    └── watch-fun/
        └── src/
            ├── acc/
            │   ├── config.rs
            │   ├── drop.rs
            │   ├── drop_entry.rs
            │   ├── gift.rs
            │   ├── mod.rs
            │   ├── package.rs
            │   ├── payment_config.rs
            │   ├── sig_receipt.rs
            │   └── user.rs
            ├── contexts/
            │   ├── admin_close_account.rs
            │   ├── admin_set_tickets.rs
            │   ├── admin_set_winners.rs
            │   ├── buy_package.rs
```

Files included in the code review

```
│   ├── buy_package_gift.rs
│   ├── claim_gift.rs
│   ├── claim_tickets.rs
│   ├── config.rs
│   ├── enter_drop.rs
│   ├── mod.rs
│   ├── operator_set_tickets.rs
│   ├── payment_config.rs
│   ├── select_winner.rs
│   ├── set_admin.rs
│   ├── set_drop.rs
│   ├── set_operator.rs
│   ├── set_package.rs
│   └── set_user.rs
├── models/
│   ├── buy_package_params.rs
│   ├── mod.rs
│   ├── sig_claim_tickets.rs
│   └── sig_package_special.rs
├── types/
│   ├── drop_error.rs
│   ├── drop_status.rs
│   └── mod.rs
├── utils/
│   ├── constants.rs
│   ├── mod.rs
│   ├── payment_validation.rs
│   └── verify_sig.rs
└── lib.rs
```

Table 1: Scope

Technical Analyses and Findings

During the Security Code Review of Watch.fun, we discovered:

- 1 finding with CRITICAL severity rating.
- 2 findings with HIGH severity rating.
- 1 finding with MEDIUM severity rating.
- 6 findings with LOW severity rating.
- 4 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

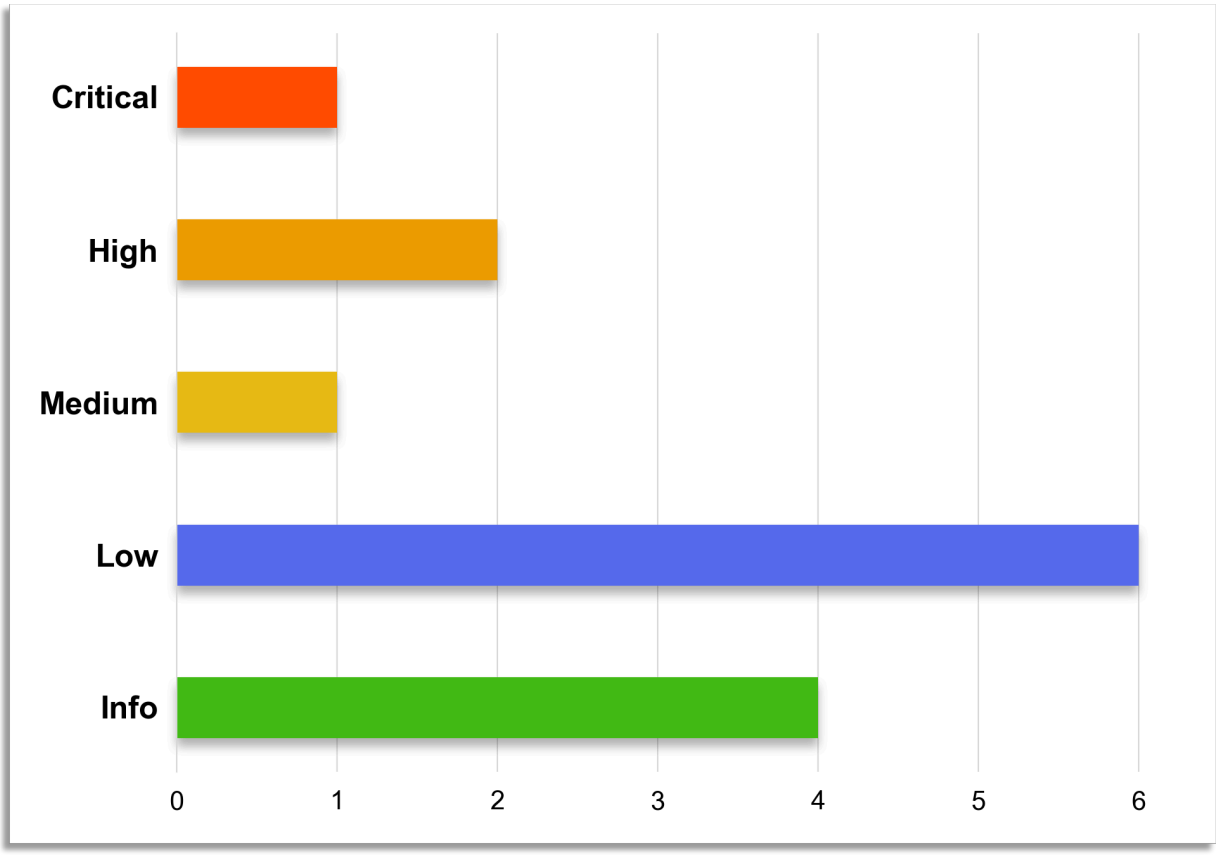

Figure 1: Findings by Severity

Findings

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description	Status
FYEO-WAT-01	Critical	Full operator signature bypass via unchecked Ed25519 headers	Remediated
FYEO-WAT-02	High	Cross-type signature replay via missing domain separation	Remediated
FYEO-WAT-03	High	special_limit bypassable	Remediated
FYEO-WAT-04	Medium	Special package buyers never receive special_bonus_ticket_count	Remediated
FYEO-WAT-05	Low	Admin mid-sale mutations can rewrite live drop and package state	Acknowledged
FYEO-WAT-06	Low	admin_close_account enables signature replay and per user cap bypass	Acknowledged
FYEO-WAT-07	Low	admin_set_winners ignores pending_winners making randomness advisory	Acknowledged
FYEO-WAT-08	Low	buy_package_gift payment signature does not bind recipient	Remediated
FYEO-WAT-09	Low	claim_gift reads a live ticket_count instead of snapshot	Remediated

FYEO-WAT-10	Low	select winner randomness handling gaps	Acknowledged
FYEO-WAT-11	Informational	DropAcc::create missing param validation for id and tickets_per_entry	Remediated
FYEO-WAT-12	Informational	No on-chain rotation path for admin, operator	Remediated
FYEO-WAT-13	Informational	Unbounded Vec pushes exceed max_len allocations	Remediated
FYEO-WAT-14	Informational	sig_receipt could be griefed	Remediated

Table 2: Findings Overview

The Classification of vulnerabilities

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets
- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys

- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Conclusion

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

Watch.fun is a Solana-based smart contract system for managing ticket-based drop events and package purchases with role-based access control. The system operates as a decentralized application that handles user ticket entries, drop prize funds, package configurations, and winner selection processes. Key functions include user account management, ticket entry validation, package purchasing with gift capabilities, administrative operations (setting winners, closing accounts, configuring drops/packages), and secure financial transactions. The system enforces strict access controls through role-based permissions (admin/operator) and uses Solana program derived addresses (PDAs) for secure state management. Protected assets include user ticket balances, drop prize funds, and package purchase payments.

Full operator signature bypass via unchecked Ed25519 headers

Finding ID: FYEO-WAT-01
Severity: **Critical**
Status: **Remediated**

Description

`parse_ed25519_ix_data` never validates `num_sigs` (data[0]) nor the three `*_ix_index` fields (bytes 4..6, 8..10, 14..16). With `num_sigs = 0`, the native Ed25519 program performs no verification yet succeeds, so an attacker can place the expected signature/pubkey/message bytes at the declared offsets and pass `verify_ed25519_signature` unconditionally. Separately, non `u16::MAX` `*_ix_index` values let the native program resolve the verified signature/pubkey/message from a different instruction (where the attacker uses their own key) while this parser reads attacker-placed bytes from the Ed25519 instruction's own data, producing a valid Ed25519 instruction paired with forged "operator-signed" payload bytes. Either gap alone allows bypass.

Proof of Issue

File name: programs/watch-fun/src/utils/verify_sig.rs
Line number: 24-56, 57-80

```rust
fn parse_ed25519_ix_data(data: &[u8]) -> Result<(&[u8], &[u8], &[u8])> {
    let header = data
        .get(..14)  // only reads 14 bytes, misses message_ix_index
        .ok_or_else(|| error!(DropError::InvalidSpecialPackageSig))?;

    // data[0] (num_sigs) - never checked, could be 0
    // data[4..6] (signature_ix_index) - never checked
    // data[8..10] (pubkey_ix_index) - never checked
    // data[14..16] (message_ix_index) - never even read

    let sig_offset = u16::from_le_bytes([header[2], header[3]]) as usize;
    let key_offset = u16::from_le_bytes([header[6], header[7]]) as usize;
    let msg_offset = u16::from_le_bytes([header[10], header[11]]) as usize;
    let msg_size = u16::from_le_bytes([header[12], header[13]]) as usize;
    // ... extracts slices at attacker-controlled offsets
}
```

Severity and Impact Summary

Externally exploitable with no privileged access. Affects every signature gated instruction: `buy_package`, `buy_package_gift`, `claim_tickets`, and special-package flows. An attacker can mint tickets and buy packages without any genuine operator signature.

Recommendation

At the start of `parse_ed25519_ix_data`, require `data[0] == 1`, `data[1] == 0`, and each of `signature_ix_index` / `pubkey_ix_index` / `message_ix_index` equals `u16::MAX`. Extend the parsed header from 14 to 16 bytes so `message_ix_index` is covered.

Cross-type signature replay via missing domain separation

Finding ID: FYEO-WAT-02
Severity: High
Status: Remediated

Description

`SigPackagePayment`, `SigPackageSpecial`, and `SigClaimTickets` are all signed by the same operator key with no domain tag in the signed message. Two concrete byte layout collisions let a signature legitimately issued for one flow satisfy verification in another: - an 8 byte special package id makes `SigPackageSpecial` byte identical to `SigClaimTickets` (attacker mints `u64::from_le_bytes(id)` tickets via `claim_tickets`) - a 36 byte special package id whose contents encode a plausible `(tx_hash, chain_id)` makes `SigPackageSpecial` byte identical to `SigPackagePayment` for an empty id not special package (attacker calls `buy_package` with no real off-chain payment).

Proof of Issue

File name: programs/watch-fun/src/models/sig_package_payment.rs, sig_package_special.rs, sig_claim_tickets.rs
Line number: 14 (payment), 12 (special), 13 (claim)

Severity and Impact Summary

Externally exploitable without any operator misbehavior, only a legitimately issued special signature and an admin chosen id of the right length and content. Results in free ticket minting or purchase of packages without the required off-chain payment. The attack surface persists as a latent footgun even with randomly chosen ids.

Recommendation

Prepend a unique single byte domain tag in each `build_message` (like `0x01` payment, `0x02` special, `0x03` claim) and mirror the change in all clients.

special_limit bypassable

Finding ID: FYEO-WAT-03
Severity: High
Status: Remediated

Description

`BuyPackageCtx` (and `BuyPackageGiftCtx`) declare `package: Account<'info, PackageAcc>` without `mut`. The handler calls `package.purchased()` which increments `special_limit_purchased`, but Anchor does not re-serialize the account at exit because it isn't writable. The counter stays at 0 forever, so `is_active()`'s `special_limit_purchased < limit` check always passes and a special package can be purchased unbounded times past its configured cap.

Proof of Issue

File name: programs/watch-fun/src/contexts/buy_package.rs
Line number: 14

```
#[account(
    seeds = [b"package", params.package_id.as_bytes()],
    bump,
)]
pub package: Account<'info, PackageAcc>,  // no `mut`, writes are dropped
```

Severity and Impact Summary

The special package limit, a core protocol invariant, is silently non-enforcing on-chain. Each purchase still requires a fresh operator signature and a unique `sig_receipt`, but nothing prevents the operator from issuing more signatures or a buyer from accumulating multiple, so the cap provides no actual ceiling.

Recommendation

Add `mut` to the account in both `BuyPackageCtx` and `BuyPackageGiftCtx` so the incremented counter persists. Consider a test that asserts `special_limit_purchased` grows across successive buys.

Special package buyers never receive special_bonus_ticket_count

Finding ID: FYEO-WAT-04
Severity: Medium
Status: Remediated

Description

The `buy_package` (and `claim_gift`) credit only `package.ticket_count` to the buyer, even when the package is marked `is_special`. The separate `special_bonus_ticket_count` field is stored on-chain during `set_packages` but is never read at purchase or claim time. Buyers who pay for and authenticate a special package receive only the base ticket amount; the special signature grants no additional tickets.

Proof of Issue

File name: programs/watch-fun/src/lib.rs
Line number: 63

```
ctx.accounts.user.add_tickets(ctx.accounts.package.ticket_count)?;
ctx.accounts.package.purchased()
```

Severity and Impact Summary

Not externally exploitable and arguably no funds at risk, but every special package buyer is silently shortchanged on the entitlement they paid for. Breaks the advertised economics of special packages and corrupts any downstream accounting that assumes buyers got what the package promised.

Recommendation

In both `buy_package` and `claim_gift`, add `special_bonus_ticket_count` (when `Some`) on top of `ticket_count` before calling `add_tickets`. Alternatively, compute total tickets once on `PackageAcc` and credit that single value.

Admin mid-sale mutations can rewrite live drop and package state

Finding ID: FYEO-WAT-05
Severity: Low
Status: Acknowledged

Description

Three related gaps in admin mutation paths: - `DropAcc::update` overwrites `end_date` and `max_entries_per_user` with no status or `entries_sold` gate, letting the admin shorten/extend the drop window or change per-user caps after entries are sold - `set_package` uses `init_if_needed` and `PackageAcc::set` unconditionally rewrites every field including resetting `special_limit_purchased = 0`, so any routine update (price, dates, metadata) silently re-opens a hit cap for another round of purchases - `PackageAcc::set` allows `is_special = true` with `special_limit = None` / `special_bonus_ticket_count = None` and `is_active()` calls `.unwrap()`, so a misconfigured special package panics with an opaque error.

Proof of Issue

File name: programs/watch-fun/src/acc/drop.rs, programs/watch-fun/src/acc/package.rs
Line number: 113, 53

```
fn update(&mut self, params: DropAccParams) -> Result<()> {
    require!(params.end_date.is_some(), DropError::DropParamMissing);
    require!(params.max_entries_per_user.is_some(), DropError::DropParamMissing);
    require!(params.metadata_hash.is_some(), DropError::DropParamMissing);

    self.end_date = params.end_date.unwrap();
    self.max_entries_per_user = params.max_entries_per_user.unwrap();
    self.metadata_hash = params.metadata_hash.unwrap();
    Ok(())
}

pub fn set(&mut self, id: String, params: PackageAccParams, bump: u8) -> Result<()> {
    // ... overwrites all fields
    self.special_limit_purchased = 0;
    self.bump = bump;
    Ok(())
}
```

Severity and Impact Summary

Admin-gated, not externally exploitable. Real rug / misconfiguration surfaces: paying users can have their drop window or package economics changed under them, special caps can be silently re-opened, and misconfigured special packages brick purchases with an opaque panic instead of a typed error.

Recommendation

Gate `DropAcc::update` on sales state. Split `PackageAcc` create vs update so existing packages only accept a narrow safe subset of fields; preserve `special_limit_purchased` across updates. In

`PackageAcc::set`, require `special_limit.is_some()` (and `special_bonus_ticket_count.is_some()`) when `is_special` is true, and replace `.unwrap()` in `is_active()` with `.ok_or(DropError::DropParamMissing)?`.

admin_close_account enables signature replay and per user cap bypass

Finding ID: FYEO-WAT-06
Severity: Low
Status: Acknowledged

Description

`admin_close_account` drains lamports and zeros data on any target account except `ConfigAcc`. Closing a `sig_receipt` or `tx_receipt` PDA re-enables replay of the special package, claim tickets, or payment signature it was meant to block. Closing a `DropEntryAcc` resets `entries_count` to 0 on next `init_if_needed`, bypassing `max_entries_per_user` and leaving `drop.entries_sold` out of sync with the user's recorded ranges.

Proof of Issue

File name: programs/watch-fun/src/lib.rs
Line number: 226

```
pub fn admin_close_account(ctx: Context<AdminCloseAccount>) -> Result<()> {
    let account = &ctx.accounts.account;
    let admin = &ctx.accounts.admin;

    let lamports = account.lamports();
    ...
    **account.try_borrow_mut_lamports()? -= lamports;
    **admin.try_borrow_mut_lamports()? += lamports;

    let mut data = account.try_borrow_mut_data()?;
    data.fill(0);

    Ok(())
}
```

Severity and Impact Summary

Admin gated and within an already broad admin trust boundary (`admin_set_tickets`, `admin_set_winners` exist). But an admin key compromise gains silent signature replay and per user cap bypass on top of direct balance manipulation, meaningfully expanding the blast radius.

Recommendation

Blocklist replay protection receipt PDAs and entry PDAs from `admin_close_account`. Either maintain an explicit allow list of closable account types or reject accounts whose seeds/discriminator match `sig_receipt`, `tx_receipt`, `DropEntryAcc`, and `GiftAcc`.

admin_set_winners ignores pending_winners making randomness advisory

Finding ID: FYEO-WAT-07
Severity: Low
Status: Acknowledged

Description

`select_winner` derives winning ticket numbers from Switchboard randomness and stores them in `drop.pending_winners`, then flips status to `WinnerPending`. `admin_set_winners` accepts a `Vec<Winner>` from instruction data and writes it verbatim via `DropAcc::set_winners`, which never reads `pending_winners`. There is no check that submitted winners match the randomness draw, that counts line up, that there are no duplicates, or that tickets are in range. The Switchboard step is therefore decorative, the admin can finalize with any ticket numbers regardless of the on-chain draw.

Proof of Issue

File name: programs/watch-fun/src/lib.rs, programs/watch-fun/src/acc/drop.rs
Line number: 247 (lib.rs), 74 (drop.rs)

```
pub fn admin_set_winners(ctx: Context<AdminSetWinnersCtx>, winners: Vec<Winner>) ->
Result<()> {
    ctx.accounts.drop.set_winners(winners)
}

pub fn set_winners(&mut self, winners: Vec<Winner>) -> Result<()> {
    self.winners = winners;
    self.status = DropStatus::Completed;
    Ok(())
}
```

Severity and Impact Summary

Not externally exploitable (admin gated), but the program documents on-chain randomness as the winner-selection mechanism and this gap makes that guarantee cosmetic. An admin key compromise turns winner selection into pure admin choice with the randomness step providing no actual constraint.

Recommendation

In `DropAcc::set_winners`, require `winners.len() == pending_winners.len()`, each submitted ticket should be contained in `pending_winners`, no duplicates, and each ticket in `1..=entries_sold`.

buy_package_gift payment signature does not bind recipient

Finding ID: FYEO-WAT-08
Severity: Low
Status: Remediated

Description

`SigPackagePayment::build_message` produces `buyer(32) + tx_hash(32) + chain_id(4) + package_id(N) + expiry(8)`. The signed message includes neither a flow discriminator (self buy vs gift) nor the intended recipient. Two considerations:
- **Flow interchangeable:** A buyer who paid off-chain for a gift can submit the same `payment_sig` through `buy_package` instead, crediting tickets to themselves. - **Recipient unbound:** In `buy_package_gift`, the payment signature is validated against `buyer`, but the gift is created for `ctx.accounts.user` (recipient). `BuyPackageGiftCtx` only constrains `user.wallet != buyer.key()`. The buyer can pass any user account and redirect the gift to an arbitrary recipient the operator never authorized.

Proof of Issue

File name: programs/watch-fun/src/models/sig_package_payment.rs, programs/watch-fun/src/lib.rs
Line number: 14 (sig_package_payment.rs), 67 (lib.rs)

```
// recipient never included
fn build_message(&self, user: &Pubkey, package_id: &String) -> Vec<u8> {
    let mut msg = Vec::new();
    msg.extend_from_slice(user.as_ref());       // buyer only
    msg.extend_from_slice(&self.tx_hash);
    msg.extend_from_slice(&self.chain_id.to_le_bytes());
    msg.extend_from_slice(package_id.as_bytes());
    msg.extend_from_slice(&self.expiry.to_le_bytes());
    msg
}

// payment_sig validated against buyer, gift goes to unbound recipient
pub fn buy_package_gift(ctx: Context<BuyPackageGiftCtx>, params: BuyPackageParams) ->
Result<()> {
    params.payment_sig.is_valid(id, ix, buyer, operator)?;  // buyer-bound only
    // ...
    let recipient = ctx.accounts.user.wallet;  // not bound by signature
    ctx.accounts.gift.set_inner(GiftAcc { gifter, recipient, package_id, bump });
}
```

Severity and Impact Summary

Externally exploitable without privileged access. Any buyer can redirect a gift to an unintended recipient or claim it as a self purchase. The operator signature provides no on-chain guarantee about who receives the tickets.

Recommendation

Include a flow discriminator byte in `build_message`, for the gift flow, include the intended recipient pubkey in the signed message. Verify it matches `ctx.accounts.user.wallet` in the handler.

claim_gift reads a live ticket_count instead of snapshot

Finding ID: FYEO-WAT-09
Severity: Low
Status: Remediated

Description

`GiftAcc` stores only `gifter`, `recipient`, `package_id`, `bump`. `claim_gift` credits `ctx.accounts.package.ticket_count` read live from the package at claim time. Combined with admin mutability of `PackageAcc`, a gift's yield can be rewritten between purchase and claim; the recipient receives whatever `ticket_count` is set to at the moment of redemption. Gifts also don't factor into any "already sold into" signal, so future guards against mid-sale mutation won't necessarily protect outstanding gifts.

Additionally, `claim_gift` does not call `package.is_active()`, unlike `buy_package` and `buy_package_gift` which both gate on it. This means a gift can be claimed after the package has expired (`end_date` passed) or its special limit has been exhausted. Combined with the live-read behavior, if an admin sets `ticket_count = 0` on an expired package (as part of a routine cleanup or reconfiguration via `set_packages`), any unclaimed gifts for that package silently yield 0 tickets. The recipient loses their entitlement with no on-chain signal that the gift was devalued. Snapshotting at purchase eliminates both the drift and the expiry concern.

Proof of Issue

File name: programs/watch-fun/src/lib.rs, programs/watch-fun/src/acc/gift.rs
Line number: 102, 5

```
pub fn claim_gift(ctx: Context<ClaimGiftCtx>, gift_id: String) -> Result<()> {
    ctx.accounts.user.add_tickets(ctx.accounts.package.ticket_count)
}

// No ticket_count snapshot stored
pub struct GiftAcc {
    pub gifter: Pubkey,
    pub recipient: Pubkey,
    pub package_id: Pubkey,
    pub bump: u8,
}
```

Severity and Impact Summary

Admin-gated, not externally exploitable. Snapshotting is the recommended fix regardless of whether package mutation ever gets guarded. Also related to the `special_bonus_ticket_count` concern: a fix for both is storing the credited amount on `GiftAcc` at purchase. The missing `is_active()` check compounds the risk, since an expired/reconfigured package can silently zero out unclaimed gifts.

Recommendation

Extend `GiftAcc` with `ticket_count` and `special_bonus_ticket_count`, populate them in `buy_package_gift`, and read them in `claim_gift`. This makes the credited amount immutable at purchase time regardless of later package mutations or expiry. Side benefit: `PackageAcc` can be removed from `ClaimGiftCtx`, shrinking its trust surface and eliminating the missing `is_active()` check as a concern entirely.

select winner randomness handling gaps

Finding ID: FYEO-WAT-10
Severity: Low
Status: Acknowledged

Description

Four independent weaknesses in how `select_winner` sources and consumes Switchboard randomness: - `reveal_slot > 0` is not a freshness check so an admin can grind across any previously revealed account - winner derivation pulls only 4 of 32 randomness bytes and has no attempt cap, so the 5th+ winners aren't independent draws and small `entries_sold` can burn CU - no Switchboard account-type discriminator check, only an owner check - both mainnet and devnet Switchboard program IDs are accepted in the same compiled binary.

Proof of Issue

File name: programs/watch-fun/src/lib.rs, programs/watch-fun/src/contexts/select_winner.rs
Line number: 178 (lib.rs), 27 (select_winner.rs)

```
pub fn select_winner(ctx: Context<SelectWinnerCtx>) -> Result<()> {
    let drop = &mut ctx.accounts.drop;
    let data = ctx.accounts.randomness.data.borrow();
    require!(data.len() >= 184, DropError::InvalidRandomnessAccount);
    let reveal_slot = u64::from_le_bytes(data[144..152].try_into().unwrap());
    require!(reveal_slot > 0, DropError::RandomnessNotReady);

    ...

    while winning_tickets.len() < winners_count {
        let base = chunks[attempt % 4] ^ chunks[(attempt + 1) % 4].wrapping_add(attempt
as u64);
        ...
        attempt += 1;
    }
}


#[account(
    constraint = (
        *randomness.owner == SWITCHBOARD_MAINNET_PID
        || *randomness.owner == SWITCHBOARD_DEVNET_PID
    ) @ DropError::InvalidRandomnessAccount
)]
pub randomness: AccountInfo<'info>,
```

Severity and Impact Summary

All four concerns are admin gated. The admin already finalizes winners via `admin_set_winners`, so no external user can exploit them. They matter as hardening against a grinding or compromised admin and as auditability improvements that make the randomness step a real trustless guarantee rather than advisory.

Recommendation

Bind `reveal_slot` to a recent slot window or to `drop.end_date`. Hash expand all 32 randomness bytes and cap collision retry attempts. Verify the Switchboard `RandomnessAccountData` discriminator. Select the Switchboard program ID at compile time with a `cfg` feature flag so each deployment only trusts its cluster-appropriate program.

DropAcc::create missing param validation for id and tickets_per_entry

Finding ID: FYEO-WAT-11
Severity: **Informational**
Status: **Remediated**

Description

Two admin supplied params lack basic validation: - `id` is never required to be non empty. The reinit guard in `set()` is `self.id.is_empty()`, so after `set_drop("")` creates a drop with an empty id, a second `set_drop("")` re-enters `create()` and wipes live state (`entries_sold`, `status`, `winners`, `pending_winners`, dates) - `tickets_per_entry` is accepted without a `> 0` check. When `tickets_per_entry == 0`, `enter_drop` charges `0 * entries = 0` tickets and the balance check always passes, allowing unlimited free entries toward winner selection.

Proof of Issue

File name: programs/watch-fun/src/acc/drop.rs
Line number: 81

```
fn create(&mut self, id: String, params: DropAccParams, bump: u8) -> Result<()> {
    require!(params.start_date.is_some(), DropError::DropParamMissing);
    require!(params.end_date.is_some(), DropError::DropParamMissing);
    require!(params.tickets_per_entry.is_some(), DropError::DropParamMissing);
    // missing: require!(!id.is_empty(), ...)
    // missing: require!(params.tickets_per_entry.unwrap() > 0, ...)

    self.id = id;  // empty string passes, reinit guard self.id.is_empty() stays true
    self.tickets_per_entry = params.tickets_per_entry.unwrap(); // 0 allowed
    ...
}
```

Severity and Impact Summary

Admin gated and not externally exploitable, but both are material footguns: the first silently destroys drop state (including completed drops), the second turns any drop into a free-for-all.

Recommendation

At the top of `DropAcc::create`, add `require!(!id.is_empty(), DropError::DropParamMissing)` and `require!(params.tickets_per_entry.unwrap() > 0, DropError::DropParamMissing)` (after the existing `is_some()` check).

No on-chain rotation path for admin, operator

Finding ID: FYEO-WAT-12
Severity: **Informational**
Status: **Remediated**

Description

`ConfigAcc` only exposes `init`; there is no setter for `admin` or `operator`. `ConfigCtx` uses Anchor's `init` and no other context takes `ConfigAcc` as `mut`. `admin_close_account` explicitly refuses `ConfigAcc`, so there is also no delete and reinitialise path. Once initialised, both privileged `Pubkey`s are immutable on-chain.

Proof of Issue

File name: programs/watch-fun/src/acc/config.rs
Line number: 5

```
pub struct ConfigAcc {
    pub admin: Pubkey,
    pub operator: Pubkey,
}

impl ConfigAcc {
    pub fn init(&mut self, admin: Pubkey, operator: Pubkey) -> Result<()> {
        self.admin = admin;
        self.operator = operator;
        Ok(())
    }
}
```

Severity and Impact Summary

Not an active vulnerability. Operational risk: a compromised or lost admin/operator key permanently bricks governance and (for operator) the ability to issue signed ticket grants. Recovery requires a program upgrade adding a rotation instruction.

Recommendation

Add an admin gated `set_config` / key rotation instruction that can update `admin` and `operator` on an existing `ConfigAcc`. Consider requiring a two step handoff for admin (current admin proposes, new admin accepts) to prevent transfer to an inaccessible key.

Unbounded Vec pushes exceed max_len allocations

Finding ID: FYEO-WAT-13
Severity: **Informational**
Status: **Remediated**

Description

Two accounts with `#[max_len]` sized `Vec` fields push without a length guard. `DropAcc::set_pending_winners` / `set_winners` push into fields sized `#[max_len(5)]`; `select_winner` already clamps at its call site but `admin_set_winners` passes the instruction argument through unvalidated. `DropEntryAcc::set` pushes into `entries_ranges` sized `#[max_len(100)]`, each `enter_drop` call appends one range regardless of `entries`, so 100 single entry calls fill the account and the 101st serialization reverts, permanently locking the user out of the drop even when `max_entries_per_user` budget remains. `enter_drop` also lacks an `entries > 0` check, so the same exhaustion is reachable at zero ticket cost via `enter_drop(0)` calls.

Proof of Issue

File name: programs/watch-fun/src/acc/drop_entry.rs, programs/watch-fun/src/lib.rs
Line number: 31 (drop_entry.rs), 124 (lib.rs)

```
// pushes without length guard
pub fn set(&mut self, params: DropEntryAccParams, bump: u8, start: u64, entries: u64) ->
Result<()> {
    if self.entries_count == 0 {
        self.create(params, bump);
    }
    self.entries_count =
self.entries_count.checked_add(entries).ok_or(DropError::Overflow)?;
    self.entries_ranges.push(TicketRange { start, count: entries }); // no len < 100
check
    Ok(())
}

// no entries > 0 check
pub fn enter_drop(ctx: Context<EnterDropCtx>, entries: u64) -> Result<()> {
    // entries = 0 passes all checks, still pushes a zero count TicketRange
    let tickets =
drop.tickets_per_entry.checked_mul(entries).ok_or(DropError::Overflow)?;
    require!(user.ticket_balance >= tickets, DropError::InsufficientTickets); // 0 >= 0
passes
}
```

Severity and Impact Summary

Both produce opaque serialization aborts instead of typed errors. Self-inflicted only. `DropEntryAcc` is scoped per `(drop, user)`, but the lockout is irrecoverable and a buggy client could trigger it.

Recommendation

Add `require!(vec.len() < N, DropError::...)` guards before each push to surface a typed error. In `enter_drop`, also add `require!(entries > 0, DropError::InvalidTicketAmount)` to close the zero cost trigger. For `entries_ranges`, prefer range merging contiguous ranges (natural given `start = entries_sold + 1`) or a `realloc` path to avoid the per user 100 transaction ceiling entirely.

sig_receipt could be griefed

Finding ID: FYEO-WAT-14
Severity: **Informational**
Status: **Remediated**

Description

`init_sig_receipt` checks `require!(receipt.lamports() == 0, ...)` and then calls `system_program::create_account`, which itself fails on a pre-funded target. The `sig_receipt` PDA is deterministic from the 64-byte operator signature, so anyone who observes that signature (via some leak) could send a few lamport to the derived address and permanently burn the grant. The legitimate call reverts with `SignatureAlreadyUsed` and the operator must re-sign. `tx_receipt` uses Anchor's `init`, which tolerates pre-funded PDAs, and is unaffected.

Proof of Issue

File name: programs/watch-fun/src/acc/sig_receipt.rs
Line number: 10

```
pub fn init_sig_receipt<'info>(
    signature: &[u8; 64],
    receipt: &UncheckedAccount<'info>,
    payer: &Signer<'info>,
    system_program: &Program<'info, System>,
    program_id: &Pubkey,
) -> Result<()> {
    let seeds: &[&[u8]] = &[b"sig_receipt", &signature[0..32], &signature[32..64]];
    let (expected_pda, bump) = Pubkey::find_program_address(seeds, program_id);

    require!(receipt.key() == expected_pda, DropError::InvalidSpecialPackageSig);
    require!(receipt.lamports() == 0, DropError::SignatureAlreadyUsed);

    // system_program::create_account also fails on pre-funded accounts
    anchor_lang::system_program::create_account(
        CpiContext::new_with_signer(/* ... */),
        lamports, space as u64, program_id,
    )?;
    Ok(())
}
```

Severity and Impact Summary

Externally exploitable griefing IF there is leakage of the signature. Affects special package purchases and `claim_tickets`. The attacker gains nothing, but denies service to specific users on specific operator issued grants; because each signature is single use by design, a griefed grant is unrecoverable without operator re-issuance.

Recommendation

Anchor's init path handles pre-funded PDAs.

Our Process

Methodology

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.

Figure 2: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.